

# Learning Speed Adaptation for Flight in Clutter

Guangyu Zhao\*, Tianyue Wu\*, Yeke Chen and Fei Gao



Fig. 1: **Flight with speed adaptation in complex natural clutter.** (Top left) The vehicle aggressively flies until it is near clutter, and further decelerates when observing a hidden bamboo behind a tree. (Top right) The vehicle flies cautiously when crossing a narrow gap between two tree trunks. (Bottom left) The vehicle flies smoothly between obstacles. (Bottom right) The vehicle flies aggressively after observing the open space in front of it.

**Abstract**—Animals learn to adapt speed of their movements to their capabilities and the environment they observe. Mobile robots should also demonstrate this ability to trade-off aggressiveness and safety for efficiently accomplishing tasks. The aim of this work is to endow flight vehicles with the ability of speed adaptation in prior unknown and partially observable cluttered environments. We propose a hierarchical learning and planning framework where we utilize both well-established methods of model-based trajectory generation and trial-and-error that comprehensively learns a policy to dynamically configure the speed constraint. Technically, we use online reinforcement learning to obtain the deployable policy. The statistical results in simulation demonstrate the advantages of our method over the constant speed constraint baselines and an alternative method in terms of flight efficiency and safety. In particular, the policy behaves perception awareness, which distinguish it from alternative approaches. By deploying the policy to hardware, we verify that these advantages can be brought to the real world.

**Index Terms**—Aerial Systems; Applications; Motion and Path Planning; Reinforcement Learning.

## I. INTRODUCTION

ANIMALS seldom move at their maximum speeds due to limited sensory, reaction, or motor capabilities. For instance, some animals, such as birds, enhance the resolution of spatial perception by slowing down while foraging [1]. Cheetahs almost never chase their prey at full speed due to the difficulties of sharp turns or footing maintenance [2]. Such compromising behaviors are especially likely to occur in constrained environments [3], where animals regulate their

speed to ensure safety considering their limited capabilities, e.g., budgerigars fly at a low speed to ensure collision-free crossing of narrow gaps [4].

The same goes for mobile robots. The limited sensory update frequency, time-consuming decision-making, and imperfect motor control capabilities, to name a few, inherently limit the allowed speed of their movements that satisfies safety regards. This efficiency-safety trade-off is always present, no matter how far a hardware or algorithmic system has evolved. Therefore, like animals, robots should be able to adaptively regulate their speed of movements based on an integrated cognition relating self-awareness, e.g., of their own capability limitations, and other-awareness, e.g., of the external environments [5].

This paper focuses on flight vehicles, where some agile behaviors have been preliminarily achieved [6]–[9]. These advances are made possible by the considerable development of model-based trajectory planners [6], [10] and controllers [7], [11] in the last decades, and the recent application of model-free learning techniques [8], [9]. However, most of the existing planning and control schemes leave the task of determining the speed constraint for the user, which is *conservatively set to constant* at deployment time [6], [12]. While some works utilize reinforcement learning (RL) to learn a policy that directly outputs low-level commands with implicit speed adaptations [8], [9], the success of these works occurs for the time being only in prior known environments. In contrast, this paper considers the problem of safe flight in unknown, partially observed, and cluttered environments.

An intuitive idea is to trial and error, learning a policy *from scratch* to enable naturally embedded speed adaptation according to the inherent limitations of the system, as implemented by a concurrent work [13]. However, such an approach is expensive to learn a deployable policy, and at this stage can only be deployed for simpler scenarios than a state-of-the-art model-based trajectory planner can handle [6], [12]. Instead, in this paper, we take advantage of the insights gained over the past decades in classical trajectory generation and

Manuscript received: March, 9, 2024; May, 25, 2024; June, 19, 2024.

This paper was recommended for publication by Editor Jens Kober upon evaluation of the Associate Editor and Reviewers' comments. This work was supported by the National Natural Science Foundation of China under grant no. 62322314. (Corresponding Author: Fei Gao)

\*Guangyu Zhao and Tianyue Wu contribute this work equally.

All authors are with the Institute of Cyber-Systems and Control, College of Control Science and Engineering, Zhejiang University, Hangzhou 310027, China, and also with the Huzhou Institute, Zhejiang University, Huzhou 313000, China.

E-mail: {zhaoguangyu, tianyueh8erobot, chenyeke, fgaoaa}@zju.edu.cn

Digital Object Identifier (DOI): see top of this page.

tracking frameworks, which are favored due to their formal safety guarantee and generalizability that is not available by policies learned from scratch.

In particular, we decompose the policy into a hierarchical one, where the outer-loop policy dynamically configures the *speed constraint* and can be effectively learned with online RL. The inner-loop policy, which is conditioned on the output of the outer-loop policy, generates the trajectory for execution. Fortunately, modern model-based trajectory planners [6], [12], [14], despite their requirement for a pre-determined constant speed constraint, have evolved considerably to serve as a near-optimum to the inner-loop policy in the scenarios they are specifically designed for. Similar hierarchical frameworks that combine experience-based learning and model-based optimization can be found in problems such as model predictive control (MPC) [15]–[17] and legged locomotion [18], [19].

Our main contribution is a system that **hierarchically combines the model-based trajectory planner and a learned outer-loop policy** to enable aggressiveness-adaptive flight in clutter. This approach is benchmarked to outperform baselines with constant constraints and an alternative approach [20]. We also demonstrate the system in real-world scenarios, including a complex natural clutter shown in Fig. 1, exhibiting aggressive but safe flight in the wild. One of the crucial technical designs to effectively learn the outer-loop policy is a two-stage reward scheme that is employed to overcome the challenge of stochasticity and sparsity posed by the early-termination penalty.

## II. RELATED WORK

### A. Adaptive Motion Planning

The works that have the most similar motivation to us are [20]–[22], which endeavor to alter flying speed with model-based motion planners. In [20], [22], the authors impose the desired adaptive behavior by incorporating the velocity into trajectory planning via handcrafted cost functions. However, these cost functions are not expressive enough to comprehensively address the problem and inevitably lead to inflexible behavior. Methodologically, while we believe that powerful modeling-based approaches can be raised through more generalized and accurate formulations and specialized computational techniques, learning-based approaches, if feasible, can serve as a simpler alternative. Zhou et al. [21] employ an online learning method based on Bayesian optimization to decide hyperparameter for trajectory planners. However, the painful convergence speed of Bayesian optimization makes their method incapable of adapting to a rapidly updating observation. We believe that an offline training mode, where the policy is determined before deployment, is more appropriate for the goal. Richter et al. [23] formalize the problem of planning in an unknown environment and indicate the fundamental intractability in this formulation is on the belief that must capture the distribution of environments. Instead of directly operating on this distribution, the authors propose to learn a collision probability based on hand-coded features, while by striking out artificial features, our method is expected to achieve more flexible pattern recognition by jointly training perception and action modules.

As mentioned in section I, a work [13] released within weeks of our submission coincidentally studies speed-adaptive flight, but follows a different idea by training a policy directly outputting acceleration commands. In this approach, speed constraint is not explicitly incorporated into the action space. A specialized latent space used to maintain historical information is the key for the policy, which is achieved by a model-based mapping algorithm [24] in our framework. Our approach is tested in clutter (see Fig. 1, Fig. 5, and Fig. 10) that are much more complex compared to those in [13]. The unique success of our approach stems not only from the learned policy, but also the generalizability and safety of the model-based trajectory planner [12].

### B. Combining Learning and Model-based Planner for Navigation in Cluttered Environments

Recently, some works enhance traditional planning and control frameworks with learned modules to achieve more robust or efficient performance. Previous works [14], [25] employ imitation learning to directly predict local waypoints from visual input without geometric mapping for perceptive navigation. RL is also used to find an optimal policy [26] or a value estimator [27] as additional modules for navigation without manual labeling. The most similar of these works to ours in terms of technical pipelines are [28], [29], which employ RL to regularize hyperparameters for a classical local planning algorithm, Dynamic Window Approach (DWA). In these works, collision penalties can be densified by relaxing them as distance to the nearby obstacles. However, in this work, the spatial distribution of trajectories generated by the planner is almost independent of the output of the learned policy, so penalizing distance-related metrics is not causally meaningful, for which we employ a two-stage reward scheme. Moreover, the vehicle in our setup also has to deal with the limited field of view (FOV), rather than the near omnidirectional perception in [28], [29], which requires it to behave in a perception-aware manner. We explicitly enable this by designing a local map representation (see section V-B). While the above works are about ground robots navigating with low speed, our work contributes to agile flight systems, which requires the RL policy to cooperate with 3D perception and a much more complex modern planner backbone. Therefore, our work has a unique application contribution.

## III. PROBLEM FORMULATION

We wish to control a flight vehicle through a prior unknown environment from one point to another with onboard perception. This problem can be formalized as a control problem in a partially observable Markov decision process (POMDP). We first overview the POMDP tuple  $(S, A, T, R, \Omega, O)$  and then formulate the problem.

- States  $S$ . The states space is divided into two parts, one is the controllable states  $S^c$  which consists of the states of the vehicle. The other part of  $S$  is the environment  $S^e$ . In the context of our problem,  $S^e$  can be described as the occupancy of the space, i.e.,  $s^e = \{0, 1\}^n$ .
- Actions  $A$ . Actions can be thought of as control commands in a general sense, which can be outputs of the trajectory planner or the actuator of the vehicle.

- Conditional transition probabilities  $T$ . According to the definition of  $S$ , the transition probability is also divided into two independent components, where we impose no assumption to the state transition of  $s^c$ , while assuming  $p(s^e | s^e, a_t) = 1$ , i.e., the environment  $S^e$  is static.
- The reward function  $R$ . As the topic of this paper suggests, our reward function is defined as *maximizing movement speed and minimizing collision loss*.
- Observations  $\Omega$ . Observations include the state estimates of the vehicle, which is assumed to be perfect, and the partial observation of the environment  $s^e$ . The latter is limited by the FOV of the external sensor (in this paper, the depth camera) and occlusion by obstacles.
- Conditional observation probabilities  $O$ . We preprocess the raw sensor input into a local occupancy map [24]. Thus, we are viewed as following the same assumptions about the conditional observation equations as [24].

The action at each moment can be viewed as generated by a policy conditioned on the current observation and goal of navigation  $g \in G$ , i.e.,  $a_t \sim \pi(\cdot | o_t, g)$ . The goal is to find the policy that maximizes the expected cumulative return,

$$\max_{\pi} \mathbb{E}_{s_{t+1} \sim p(\cdot | s_t, a_t), a_t \sim \pi(\cdot | o_t, g), g \sim p_g} \left[ \sum_t \gamma^t r(s_t, a_t, g) \right], \quad (1)$$

where  $p_g$  is the distribution of  $g$ , and  $\gamma$  is the discount factor.

## IV. FRAMEWORK

Our framework involves a learned policy, which is denoted as  $\pi^\dagger$ , a model-based trajectory planner, and the standard perception and low-level control ends. The policy  $\pi^\dagger$  and model-based trajectory planner form a hierarchical policy to optimize (1).

### A. Model-based Trajectory Planner

The trajectory planner receives the local geometric map and the states of the vehicle as inputs and generate a collision-free trajectory. The planner generates the trajectory by solving a constrained optimization in the form of

$$\min_{u(t), T} \int_0^T J(u(t)) + \rho(T), \quad (2a)$$

$$\begin{aligned} &\text{s.t.} \\ &u(t) \in \mathcal{F}, \quad \forall t \in [0, T], \end{aligned} \quad (2b)$$

$$\mathcal{G}(u(t)) \preceq \mathbf{0}, \quad \forall t \in [0, T], \quad (2c)$$

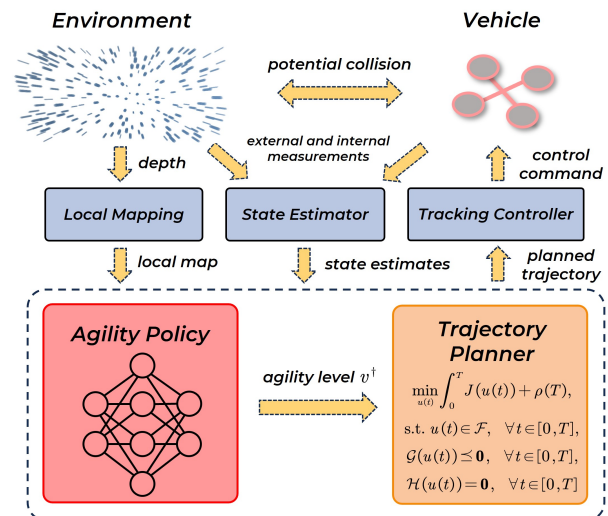
$$\mathcal{H}(u(t)) = \mathbf{0}, \quad \forall t \in [0, T], \quad (2d)$$

where  $u(t)$  is the planning quantities,  $\rho : [0, \infty) \mapsto [0, \infty]$  the time regularization term,  $\mathcal{F}$  the collision free region,  $\mathcal{G}(\cdot)$  the dynamic constraints imposed by the physical limitation of the vehicle or users, and  $\mathcal{H}(\cdot)$  summarizes the equality relations between the planning quantities and the states or control commands [30].

In the problem of navigation in cluttered environments, an inequality in (2c) represents the level of aggressiveness on speed of vehicle  $v$ , such as  $\|v\| \leq \bar{v}$  or  $v \preceq \bar{v}$  [6], [12]. For the generality of our description, we write such constraints uniformly as

$$h(\mathbf{v}) \preceq v^\dagger \quad (3)$$

without assigning them specific forms, where  $v^\dagger$  is predetermined as the planner solving for a trajectory.



**Fig. 2: Overview of the system with the hierarchical policy.**

### B. Hierarchical Policy Optimization

Traditionally,  $v^\dagger$  is conservatively determined in advance based on the user’s observations of the environment to ensure safety. However, such an approach does not reasonably take advantage of the performance of the hardware and algorithmic systems. Instead, jointly optimizing the speed constraint and trajectory to optimize (1) can further unlock the potential of the system.

We define a decomposition of the policy  $\pi'$  which augments  $\pi$  in (1) with an auxiliary action  $v^\dagger$

$$\pi'(a'|o, g) := \pi'(a, v^\dagger|o, g) = \pi''(a|v^\dagger, o, g) \cdot \pi^\dagger(v^\dagger|o, g), \quad (4)$$

where  $\pi'$  is decomposed as two hierarchical policies  $\pi''$  and  $\pi^\dagger$ , the former of which is conditioned on the auxiliary action while the latter decides it. The action space of the original POMDP is augmented as a result of the introduction of an auxiliary action, i.e.,  $A' = A \times V^\dagger$ . However, no other part of the POMDP is changed, so we can define a *hierarchical policy optimization* problem

$$\begin{aligned} & \max_{\pi'', \pi^\dagger} \mathbb{E}_{v_t^\dagger \sim \pi^\dagger(\cdot | o_t, g), a_t \sim \pi''(\cdot | v_t^\dagger, o_t, g)} \left[ \sum_t \gamma^t r(s_t, a_t, g) \right] = \\ & \max_{\pi^\dagger} \max_{\pi''} \mathbb{E}_{v_t^\dagger \sim \pi^\dagger(\cdot | o_t, g), a_t \sim \pi''(\cdot | v_t^\dagger, o_t, g)} \left[ \sum_t \gamma^t r(s_t, a_t, g) \right], \end{aligned} \quad (5)$$

equivalent to (1), where, for more concise writing, we omit the state and goal distributions in (1). This problem is equivalent to (1) in the sense that if  $\pi$  and  $\pi'$  impose the same distribution on  $a$  when conditioned on any  $o \in O$  and  $g \in G$ , the objective function values of (1) and (5) are consistent.

Model-based trajectory planners [6], [10], [12], [14], after decades of development, *can be regarded as a good approximation of the optimal  $\pi''$*  conditioned on a given  $v^\dagger$ . Therefore, only the outer-loop policy  $\pi^\dagger$  needs to be learned. We use online model-free RL to learn this policy, which is detailed in the next section.

### C. System Overview

We conclude this section with an overview of the whole system, as illustrated in Fig. 2.

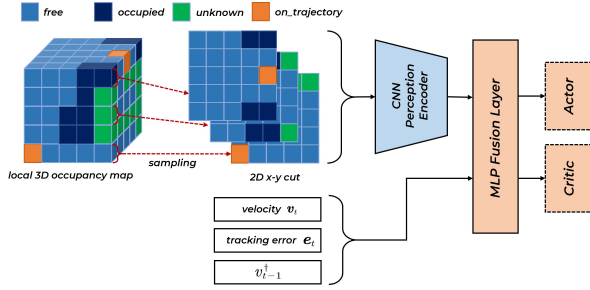


Fig. 3: **Illustration of the policy architecture and observation implementation.** The figure shows the network architecture of policy, where actor and critic share the CNN encoder and fusion layer. We also highlight the 3D occupancy map where each cell is assigned a state, and its x-y profiles are sampled as the input of the network.

At the uppermost end of the system, the vehicle obtains partial observations of the environment via external sensors, which are used to provide state feedback (e.g., via the visual-inertial odometry (VIO)) and to construct local occupancy maps (e.g., from the depth camera). The state estimates and local map are fed to the policy  $\pi^\dagger$  and trajectory planner. We choose to use the occupancy map as the input of the RL policy instead of the raw sensory data because the training is performed in simulation, and such an approach deals with sensor noise through a model-based approach, i.e., the employed mapping algorithm, thus bridging the gap between simulation and reality in RL training.

At each time step  $t$ , the policy  $\pi^\dagger$  outputs a high-level planning instruction  $v_t^\dagger$ . The trajectory planner generates a trajectory  $u$  in a horizon  $[t, t + T]$  conditioned on  $v_t^\dagger$ , which is treated as the action  $a_t$  of the POMDP. The trajectory is tracked by a low-level controller and executed by the vehicle's inner-loop controller. In our definition, the unknown environment as well as tracking and actuating controllers are treated as dynamics of the POMDP.

## V. REINFORCEMENT LEARNING FOR THE OUTER-LOOP POLICY

In this section, we detail how to effectively learn the outer-loop policy. Eq. (5) implies an approach to learn this policy, which treats also the trajectory planner as part of the environment dynamics. An auxiliary POMDP is considered to learn  $\pi^\dagger$ , in which the action space is  $V^\dagger$ . In the following subsections, we present the implementation of each part of the RL policy.

### A. Policy Representation

We represent policy and value functions using a branch of partially shared neural networks, as illustrated in Fig. 3. The networks share a perception encoder consists of convolution neural networks (CNNs) and a fusion layer implemented with multilayer perceptron (MLP). We implement the CNN encoder with 4 layers whose [channel number, kernel size, stride, padding] are [5,5,3,2], [32,3,2,1], [48,3,2,1] and [64,3,1,0]. The result of the CNN encoder is flattened and concatenated into the inputs of the MLP fusion layer. The output dimension of the MLP layer is 64. The actor and critic networks are both implemented as MLPs with 2 hidden layers having 64 and 32 units, respectively.

### B. Observation Space

As implied by (5), in principle,  $\pi^\dagger$  should be conditioned on the observations and the goal of navigation. However, for more effective learning, we design a specialized input space. As illustrated in Fig. 3, the input consists of the 3D local occupancy map  $m_t$ , the current velocity  $v_t$ , the current tracking error  $e_t := p_t - \hat{p}_t$ , where  $p_t$  and  $\hat{p}_t$  are the true and desired position of the vehicle, respectively, the decision at the last time step  $v_{t-1}^\dagger$ , and a *pre-planned* trajectory at the current time step generated according to  $v_{t-1}^\dagger$  and  $o_t$ , which will not be executed by the controller.

The pre-planned trajectory serves two purposes. First, it expresses the current position of the vehicle in the local map as well as the local target chosen by the trajectory planner according to the global goal  $g$ . Second, since the speed component of the trajectory does not have a significant effect on the spatial distribution of the trajectory, we can use this pre-planned trajectory as a spatial approximation of the trajectory to be planned, and therefore alert the perceptual part of the policy as to which regions in the local map are important.

We correlate the trajectory and the 3D local map by 'drawing' the former on the latter, as illustrated by Fig. 3. Specifically, each cell in the occupancy map is assigned one of four states: free, occupied, unknown, and on\_trajectory. The state unknown is set to trigger perception-aware behaviors. For more efficient and easier learning, we sample the trajectory with time interval  $\delta t$ , keeping only the x-y 2D cuts of the 3D map corresponding to the sampled points on the trajectory as input to the network, which is illustrated in Fig. 3.

### C. Early Termination

We define two cases of termination. One is when the vehicle is in an unsafe state, i.e., the vehicle is judged in the emergency\_stop or collided state, which is determined by the state machine in a modern motion planning system [6], [12]. The other occurs when the planning solution procedure does not finish in a valid time, which is often due to too high level of aggressiveness allowed in the previous time steps, so that the vehicle is too close to a suddenly appearing obstacle under the perception latency and thus the planner fails to plan a feasible trajectory.

### D. Reward Function

As described in section III, we design the reward function to maximize traversal speed and minimize collision loss. For practical considerations, we further divide the reward function into four parts: the speed term, as follows:

$$r = r_{\text{speed}} + r_{\text{smoothing}} + r_{\text{error}} + r_{\text{danger}}, \quad (6)$$

where the specific forms of the smoothing, tracking error and collision terms are

$$r_{\text{smoothing}} = -\lambda_{\text{smoothing}} \|v_t^\dagger - v_{t-1}^\dagger\|^2, \quad (7)$$

$$r_{\text{error}} = -\lambda_{\text{error}} \min \{\|e_t\|, e_{\text{max}}\}^2, \quad (8)$$

$$r_{\text{danger}} = \begin{cases} -\lambda_{\text{danger}} \|\mathbf{v}_t\|^2 & (\text{when the episode is terminal}) \\ 0 & (\text{otherwise}) \end{cases}, \quad (9)$$

for some  $\lambda_{\text{smoothing}} > 0$ ,  $\lambda_{\text{error}} > 0$ , and  $\lambda_{\text{danger}} > 0$ .

However,  $r_{\text{danger}}$  is highly sparse and stochastic, where the stochasticity is derived from the complex modern trajectory planner. This distribution of reward can cause difficulties when learning the policy with general RL algorithms. One way to relax the early-termination penalty and make it denser is to add the distance to obstacles into the reward function. However, although this may be effective for an end-to-end policy that directly outputs a low-level command [13], this kind of geometric metric cannot evaluate the risk of early termination in our framework, as the spatial distribution of the vehicle is mainly determined by the planned trajectory where smaller distances to obstacles do not necessarily imply that collisions are more likely to occur [12]. In principle, we should only settle for penalizing as an episode does terminate, which is causally relevant to some implicit factors such as perception latency and tracking failure that make overly aggressive trajectories prone to cause collisions.

Our key finding here is that with a two-stage reward, learning can take place efficiently and effectively. The only term that differs between the first and second stages is  $r_{\text{speed}}$ , which is detailed in the following.

1) *A human knowledge-based, dense reward function for pre-training*: The speed reward in the first stage, where  $\mathbf{v}_t^\dagger$  is imposed to be larger or equal to a certain constant value when training (e.g., 1m/s), is defined as follows:

$$r_{\text{speed}} = \begin{cases} \lambda_{\text{speed}}^1 (\phi_t^1 - \|\mathbf{v}_t\|) & (\phi_t^2 > \lambda_{\phi}^1) \\ \lambda_{\text{speed}}^2 (\|\mathbf{v}_t\| - \phi_t^1) & (\phi_t^2 < \lambda_{\phi}^2) \\ \lambda_{\text{speed}}^3 \|\mathbf{v}_t\| & (\text{otherwise}) \end{cases}, \quad (10)$$

where  $\lambda_{\text{speed}}^i > 0$  for  $i \in \{1, 2, 3\}$  and  $\lambda_{\phi}^i$  for  $i \in \{1, 2\}$  are hyperparameters such that  $\lambda_{\text{speed}}^2 > \lambda_{\text{speed}}^3$ , and  $\phi_t^i$ s, for  $i \in \{1, 2\}$ , are handcrafted feature values encoding the obstacle distribution. Specially,  $\phi_t^i$ s are normalized linear combinations of the distance to the nearest obstacle, volume of the obstacle, and number of obstacles at current time.

In (10),  $\phi_t^1$  is designed to shape the reward function smoother with a zero mean and  $\phi_t^2$  is a naive estimate of the level of danger by human knowledge. Therefore, the first case in (10) serves as a *dense smoothing* of the collision penalty, which imposes the policy to behave conservatively when the observed environment is manually evaluated as dangerous. Such an approach prevents the policy from being single-mindedly greedy to gain instant reward by accelerating. Note that we do not necessarily carefully tune the hyperparameters in (10), but leave the task of further optimization to the objective reward in the second stage.

2) *An objective reward for fine-tuning*: In the second stage, we restore the original intent of the speed reward, which is only to encourage greater aggressiveness:

$$r_{\text{speed}} = \lambda_{\text{speed}}^3 \|\mathbf{v}_t\|. \quad (11)$$

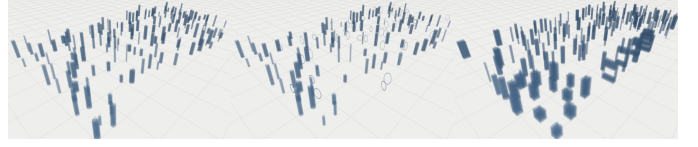


Fig. 4: **Example illustration of environments for training.** Areas in blue represent the space filled with obstacles.

## VI. TRAINING SETUP AND IMPLEMENTATION

### A. System Setup

We choose a state-of-the-art model-based trajectory planner, EGO-planner [12], as the backbone. The speed constraint is imposed in the form of  $\|\mathbf{v}\| \leq \bar{v}$ . The local mapping algorithm is implemented based on [24]. The vehicle is equipped with a forward-facing camera that can measure depth with a FOV of  $87^\circ$  (horizontal)  $\times$   $58^\circ$  (vertical) and a valid/trusted depth range from 0.28m to 5m. The refreshing frequency of the depth data is 15 Hz. A PID controller running at 50 Hz is used for position tracking.

The outer-loop policy runs at 10 Hz. Conceptually, trajectory planning is required after each outer-loop policy output, but too frequent replanning increases the situations of triggering the `emergency_stop` state in practice, as well as increasing the computational overhead. Therefore, we design a criterion to determine whether replanning is imposed by the outer-loop policy output, and if this criterion is not met, the planner follows its original implementation of replanning rules [12]. In particular, if  $\mathbf{v}_t^\dagger - \mathbf{v}_{t-1}^\dagger \in [-0.3, 0.5]$  m/s, we do not impose a replanning of the trajectory planner.

### B. Training Environment

We train the policy in a customized simulator, which parallelizes multiple (30 in our implementation) separate environments. Unlike general simulation environments, the simulator does *not* stop the clock while solving for the action, i.e., mapping and trajectory planning, in our simulation environment. Meanwhile, the sensory data is updated *asynchronously* with the action. These specialized designs are elaborated to simulate one of the most important factors limiting allowed aggressiveness, the perception latency, i.e., the time interval between perception and execution of an action. Even with these designs, however, we can only approximate perception latency because the computing platform at the deployment time is different from the one at the time of training. Another crucial factor that limits aggressiveness, the error of trajectory tracking due to limited performance of controller or possible dynamically infeasible trajectories, is also embedded in the environment by setting up the actual PID tracking controller for the vehicle, as described earlier.

### C. Training Implementation

The policy is learned with the soft actor-critic (SAC) algorithm [31]. Prioritized experience replay (PER) [32] is also applied to mitigate the sparse failure mode problem in a general way. We train the policy in multiple scenarios with variable obstacle distributions, some of which are shown in Fig. 4. We *freeze* the CNN encoder in the second stage

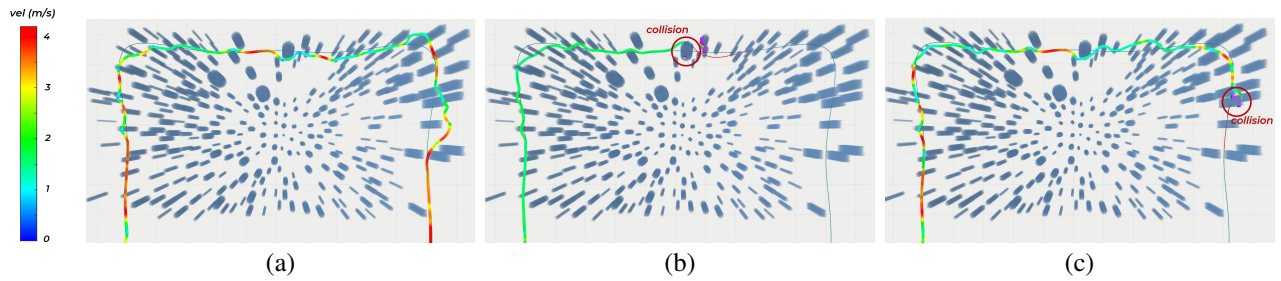


Fig. 5: **Velocity distribution along the trajectory in different setups.** The dark green curves are the reference trajectory. The colorful curves are the trajectory that the vehicle passes over, where the color represents the velocity. (a) An example result of the proposed approach. (b) An example result of the constant speed constraint  $v^\dagger = 2\text{m/s}$ . (c) An example result of the intermediate model before fine-tuning.

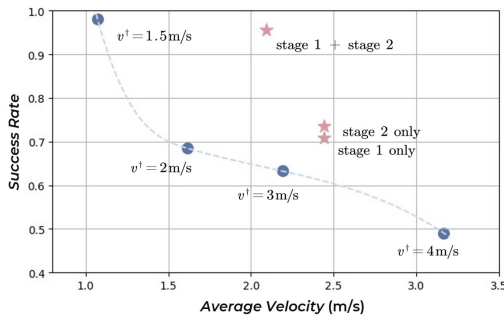


Fig. 6: **Statistical results of different setups.** Success rates are computed on 50 trials for each setup. The light blue dashed lines connect the statistics at different levels of constant speed constraint, indicating the inherent capability of the system. Average velocities are computed as the average of success trials in the 50 trials.

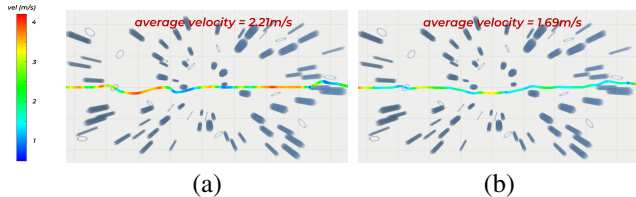


Fig. 7: **Comparison of the proposed approach and EVA-planner.** The colorful curves are the trajectory that the vehicle passes over, where the color represents the velocity. The marked average velocity is computed on 30 trials. (a) Illustration of the trajectory generated by the proposed approach. (b) Illustration of the trajectory generated by EVA-planner.

of training, considering that the first-stage reward design encourages the encoder to extract features that can flexibly respond to different patterns of the obstacle distribution.

## VII. EXPERIMENTS

### A. Simulation Experiments

We benchmark three approaches in the simulation experiments: (i) the proposed 'learned policy + model-based planner backbone', (ii) the backbone planner EGO-planner with constant speed constraint (i.e., constant  $v^\dagger$ ), and (iii) the handcrafted cost function-based (non-learning) environmental adaptive planner EVA-planner [20], where the default parameters in the cost function are used, and all parameters that EGO-planner shares with it are set equal to those of EGO-planner.

1) *Comparison of the proposed framework with constant speed constraint baselines and ablation tests:* The first two setups are evaluated in a challenging environment as shown in

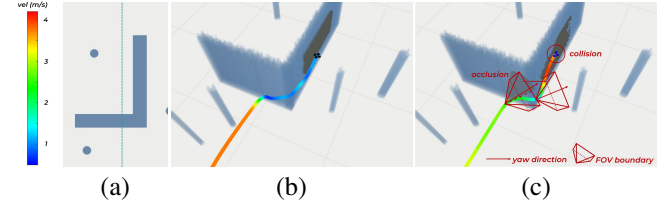


Fig. 8: **Illustration of perception-aware behaviors of the learned policy and comparison with EVA-planner.** The colorful curves are the trajectory that the vehicle passes over, where the color represents the velocity. The dark gray grid is the built local map. (a) Illustration of the scenario, where the dark green line is the reference trajectory. (b) Behaviors of the proposed approach. (c) Behaviors of EVA-planner, where the yaw angles and FOV boundaries are shown to explain the failure.

Fig. 5, where obstacles of different densities, sizes, and appearances are unevenly distributed. The evaluation environment is never seen when training. In Fig. 6, we report the statistical results of traversal velocity in success cases and the success rate which is defined as the proportion of trials that complete the entire track. A trial is judged to be 'terminal' with the same criteria as during training. In Fig. 5, we visualize the trajectories and velocities at every moment of the proposed approach and the baselines. Since the trajectory generation backbone of EVA-planner cannot always effectively generate feasible trajectory in such a complex environment, it is not fair to include it in this statistical result, so we ignore EVA-planner in Fig. 6.

The statistics show that as  $v^\dagger$  is set higher, the success rate decreases, creating a sloping downward performance curve in Fig. 6. Such performance curves capture the inherent performance of the integration of physical (e.g., sensors equipped) and algorithmic (e.g., the mapping, planning and control framework) systems. Although under the performance constraints of the system, by dynamically configuring the speed constraint, the vehicle combines the exploitation of agility and safety guarantees. It can be seen in Fig. 6 that the policy learned with both the reward stages achieves similar success rate as the lowest speed case where  $v^\dagger = 1.5\text{m/s}$ , but also exhibits considerable traversal speeds to efficiently complete the track. In contrast, a relatively aggressive constant speed constraint  $v^\dagger = 2\text{m/s}$  can frequently lead to emergency stops or collisions in areas with dense obstacles or local occlusion, as shown in Fig. 5b.

In Fig. 6, we also show the performance of the policies trained only with either the first stage (reward (10)) or the

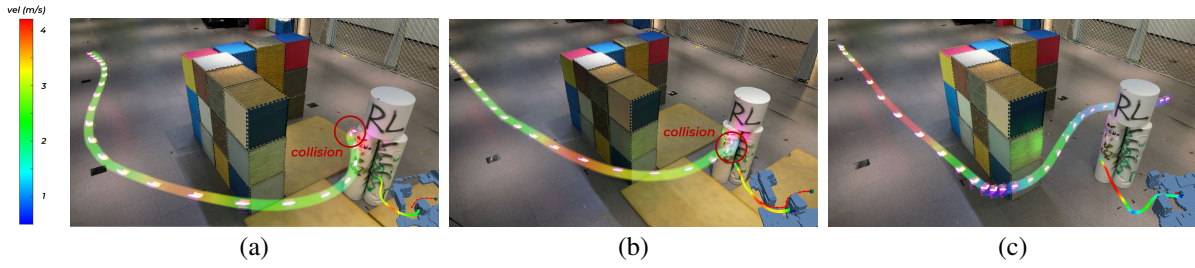


Fig. 9: **Flight to avoid a wall-like obstacle with a hidden pillar.** The planned trajectory (red curve), the executed trajectory (colorful curve), and the local map (blue) are indicated in the bottom right corner of each figure. (a) Flight with constant speed constraint  $v^\dagger = 2.5\text{m/s}$ . (b) Flight with constant speed constraint  $v^\dagger = 3.5\text{m/s}$  (c) Flight with speed adaptation.

second stage (reward (11)). The results indicate that although the final policy can successfully improve the overall performance of the system, the policy trained with only the first stage does not exhibit such improvement. This result is not so surprising since the reward scheme of the first stage of training does not reflect the objective values and impose inaccurate human knowledge. On the other hand, the necessity of the first training stage is reflected by the inferiority of the performance rendered by the policy trained directly with sparse (but objective) rewards in Fig. 6.

2) *Comparison of the proposed framework with EVA-planner and an example case of perception-aware behaviors:* EVA-planner tends to plan (sometimes overly) conservative velocities, which can be seen in Fig. 7b, while the proposed approach makes better use of the vehicle's maneuverability, at the cost of large changes in velocity, as seen in Fig. 7a.

More importantly, since the hand-designed cost function of EVA-planner does not capture some aspects of the problem, such as the necessity of perception awareness, an unsafe decision may be output by the planner. An example is illustrated in Fig. 8. In such a case, the outer-loop policy learns to behave cautiously when the vehicle plans to turn its head into area out of its perception range, which is caused by occlusion of a corner and attitude angle changes. In contrast, EVA-planner plans an unsafe velocity due to its illusion that there are no obstacles nearby, which is caused by occlusion, yaw angle changes, and perception latency. However, such perception-aware behaviors does not always occur in the policy obtained using only the first stage of reward. An example is shown in Fig. 5c, where acceleration at a corner causes the vehicle to collide.

## B. Real-World Experiments

We deploy the policy on a micro drone with a size of  $17\text{cm} \times 17\text{cm} \times 10\text{cm}$ . The drone is equipped with an onboard RealSense D430 depth camera. Computation is performed on an onboard Jetson Orin NX module. We use the NOKOV motion capture system and a VIO to obtain the state estimates in indoor and outdoor scenarios, respectively. The parameter setup is aligned with that in section VI-A.

1) *Wall-like obstacles with an obstacle hidden behind:* We first test the policy in a representative scenario where a wall-like obstacle blocks near half of the vehicle's FOV, and only after the vehicle avoids the corner could it observe the obstacle immediately afterward.

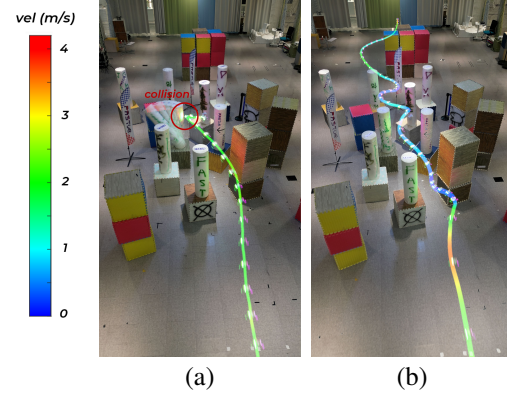


Fig. 10: **Flight through dense and large-sized obstacles.** (a) Flight with constant speed constraint  $v^\dagger = 2.5\text{m/s}$ . (b) Flight with speed adaptation.

We find that, in such a scenario, the policies under constant speed constraints, e.g.,  $v^\dagger = 2.5\text{m/s}$  and  $v_t^\dagger = 3.5\text{m/s}$  in Fig. 9, can plan a collision-free trajectory in most of the trials. However, the mapping and planning pipeline may exhibit a significant perception latency and the planner may generate a trajectory that is not dynamically feasible. As a result, the planned trajectory, which requires sudden changes of motion, can be far from being able to be perfectly tracked by the low-level controller. We note that, typically, the trajectory is dynamically feasible and thus can be well-tracked, while in some emergency cases, the trajectory optimization method of EGO-planner [12, Eq. (8)], which uses penalties to surrogate constraints for computational efficiency, can sacrifice dynamical feasibility to ensure a collision-free trajectory.

In contrast, as shown in Fig. 9c, after plugging the learned policy, the vehicle exhibits perception-aware behaviors, slowing down at the corner of the wall and recovering aggressiveness as more extents of the hidden obstacle is observed, until it eventually slows down to stop at the goal.

2) *Artificial Clutters:* We evaluate the policy in an artificial scenario shown in Fig. 10. In this scenario, a dense clutter and a large-sized obstacle exist in the first and second half of the scene, respectively. The constant speed constraint policies, as shown in the Fig. 10a, fails in half of the trials due to planning time overruns, the occurrence of emergence stops (as explained in section V-C), and also collisions caused by the same reason as that described in the previous subsection. With the learned module, the policy reasonably exhibits aggressiveness in open areas, and also behaviors cautiously among dense obstacles, as shown in Fig. 10b, thus ensuring success in almost all trials.

3) *Natural Clutters*: We also evaluate the policy in natural clutters as snapshoted in Fig. 1, where the highly uneven environment triggers rich patterns of vehicle's behavior. Despite the particularly complex environment, our policy exhibits aggressive behavior in (relatively) open areas (Fig. 1, top left and bottom right), but stays cautious to ensure safety when crossing narrow gap (Fig. 1, top right) or among obstacles (Fig. 1, bottom left).

## VIII. CONCLUSION

We propose a hierarchical learning and planning framework for aggressiveness-adaptive flight in cluttered environments. On the one hand, the hierarchical framework allows the system to obtain a strong overall performance thanks to the existing powerful model-based trajectory planner. On the other hand, by doing so, we complement the 'missing jigsaw puzzle' in traditional trajectory planners, thus unlocking the potential of the system while freeing humans from hand-tuning labor.

The main limitation of the current system is that the spatial distribution of the vehicle is determined by the trajectory planner itself, and thus in some adversarial environments the sub-optimality of the design (e.g., choice of topology) in the planner backbone may largely limit the allowed aggressiveness in the environment. To achieve versatile aggressive flight in any environment, one solution is to give greater dominance to the learned policy while still taking advantage of safety guarantees and generalizability rendered by the traditional planner. However, when the search space is expanded, learning can take place less efficiently especially when embedded in a model-based planner where the depth rendering and simulating the system are both time-consuming. Moreover, the specialized reward scheme used in this work may not be generalized when considering a different auxiliary action space. Therefore, both scalable training environment and a unified method for effective learning are highly desired.

## REFERENCES

- [1] T. Yoon, R. B. Geary, A. A. Ahmed, and R. Shadmehr, "Control of movement vigor and decision making during foraging," *Proceedings of the National Academy of Sciences*, vol. 115, no. 44, pp. E10476–E10485, 2018.
- [2] A. M. Wilson, J. Lowe, K. Roskilly, P. E. Hudson, K. Golabek, and J. McNutt, "Locomotion dynamics of hunting in wild cheetahs," *Nature*, vol. 498, no. 7453, pp. 185–189, 2013.
- [3] R. Wheatley, M. J. Angilletta Jr, A. C. Niehaus, and R. S. Wilson, "How fast should an animal run when escaping? an optimality model based on the trade-off between speed and accuracy," *Integrative and comparative biology*, vol. 55, no. 6, pp. 1166–1175, 2015.
- [4] P. Henningsson, "Flying through gaps: how does a bird deal with the problem and what costs are there?" *Royal Society Open Science*, vol. 8, no. 8, p. 211072, 2021.
- [5] J. D. Smith, W. E. Shields, and D. A. Washburn, "The comparative psychology of uncertainty monitoring and metacognition," *Behavioral and brain sciences*, vol. 26, no. 3, pp. 317–339, 2003.
- [6] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3529–3536, 2019.
- [7] M. O'Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, "Neural-fly enables rapid learning for agile flight in strong winds," *Science Robotics*, vol. 7, no. 66, p. eabm6597, 2022.
- [8] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Science Robotics*, vol. 8, no. 82, p. eadg1462, 2023.
- [9] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.
- [10] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 3259–3278, 2022.
- [11] S. Sun, A. Romero, P. Foehn, E. Kaufmann, and D. Scaramuzza, "A comparative study of nonlinear mpc and differential-flatness-based control for quadrotor agile flight," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3357–3373, 2022.
- [12] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "Ego-planner: An esdf-free gradient-based local planner for quadrotors," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 478–485, 2020.
- [13] H. Yu, C. De Wagter, and G. C. de Croon, "Mavrl: Learn to fly in cluttered environments with varying speed," *arXiv preprint arXiv:2402.08381*, 2024.
- [14] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Science Robotics*, vol. 6, no. 59, p. eabg5810, 2021.
- [15] J. Sacks and B. Boots, "Learning to optimize in model predictive control," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 10 549–10 556.
- [16] Y. Song and D. Scaramuzza, "Policy search for model predictive control with application to agile drone flight," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2114–2130, 2022.
- [17] A. Romero, S. Govil, G. Yilmaz, Y. Song, and D. Scaramuzza, "Weighted maximum likelihood for controller tuning," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 1334–1341.
- [18] Y. Yang, G. Shi, X. Meng, W. Yu, T. Zhang, J. Tan, and B. Boots, "Cajun: Continuous adaptive jumping using a learned centroidal controller," *arXiv preprint arXiv:2306.09557*, 2023.
- [19] F. Jenelten, J. He, F. Farshidian, and M. Hutter, "Dtc: Deep tracking control," *Science Robotics*, vol. 9, no. 86, p. eadh5401, 2024.
- [20] L. Quan, Z. Zhang, X. Zhong, C. Xu, and F. Gao, "Eva-planner: Environmental adaptive quadrotor planning," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 398–404.
- [21] X. Zhou, C. Xu, and F. Gao, "Automatic parameter adaptation for quadrotor trajectory planning," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 3348–3355.
- [22] L. Wang and Y. Guo, "Speed adaptive robot trajectory generation based on derivative property of b-spline curve," *IEEE Robotics and Automation Letters*, vol. 8, no. 4, pp. 1905–1911, 2023.
- [23] C. Richter, W. Vega-Brown, and N. Roy, "Bayesian learning for safe high-speed navigation in unknown environments," *International Symposium of Robotic Research (ISRR)*, pp. 325–341, 2015.
- [24] H. Moravec and A. Elfes, "High resolution maps from wide angle sonar," in *Proceedings. 1985 IEEE international conference on robotics and automation (ICRA)*, vol. 2. IEEE, 1985, pp. 116–121.
- [25] S. Bansal, V. Tolani, S. Gupta, J. Malik, and C. Tomlin, "Combining optimal control and learning for visual navigation in novel environments," in *Conference on Robot Learning (CoRL)*. PMLR, 2020, pp. 420–429.
- [26] A. Faust, K. Oslund, O. Ramirez, A. Francis, L. Tapia, M. Fiser, and J. Davidson, "Prm-rl: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 5113–5120.
- [27] D. Shah, A. Bhorkar, H. Leen, I. Kostrikov, N. Rhinehart, and S. Levine, "Offline reinforcement learning for visual navigation," *Conference on Robot Learning (CoRL)*, 2022.
- [28] M. Dobrevski and D. Skočaj, "Adaptive dynamic window approach for local navigation," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 6930–6936.
- [29] Z. Xu, G. Dhamankar, A. Nair, X. Xiao, G. Warnell, B. Liu, Z. Wang, and P. Stone, "Applr: Adaptive planner parameter learning from reinforcement," in *2021 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2021, pp. 6086–6092.
- [30] D. Bertsekas, *Dynamic programming and optimal control*. Athena scientific, 2012, vol. 4.
- [31] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning (ICML)*. PMLR, 2018, pp. 1861–1870.
- [32] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *arXiv preprint arXiv:1511.05952*, 2015.